INTERIOFLOW

System Design & Architecture

Interior Design Consulting, Project Management & Home Décor Shopping Platform

Architecture brief for implementation with Claude

Prepared 26 October 2026 • Target deployment: Vercel

Core principle: interior design consulting and project management remain the primary product. The home décor store is a complementary module.

1. Executive summary

InterioFlow is a role-based web application that brings together design consultation requests, proposals and quotations, interior project tracking, contractor task updates, and a direct home décor product store. It is designed as a modular full-stack application rather than a collection of separate services.

Recommended baseline: Next.js 16.x App Router, React 19.3.x, TypeScript, Tailwind CSS 4.x, Clerk for authentication, Supabase PostgreSQL and Storage, Razorpay for product checkout, and Vercel for hosting. Pin compatible stable package versions in a lockfile and re-check current official documentation before installation.

The Next.js 16 line is in Active LTS; React's official version page lists 19.3 as latest; Vercel documents first-class Next.js deployment and Node.js 24.x as an available default runtime. These versions should still be checked at implementation time.

2. Scope and user roles

Roles

• Customer — submit design requests, review/accept proposals, follow own projects, shop décor products, pay, and view own orders.

• Interior Designer — review assigned requests, create proposals, manage projects and milestones, assign tasks, and post progress updates.

• Contractor — view assigned tasks and update task status/notes/photos; no access to unrelated projects or store administration.

• Admin — manage roles, products, stock, orders, payment records, reports, and platform operations.

Explicit exclusions for the first release

• No vendor/supplier role, supplier dashboard, supplier quotations, or purchase orders.

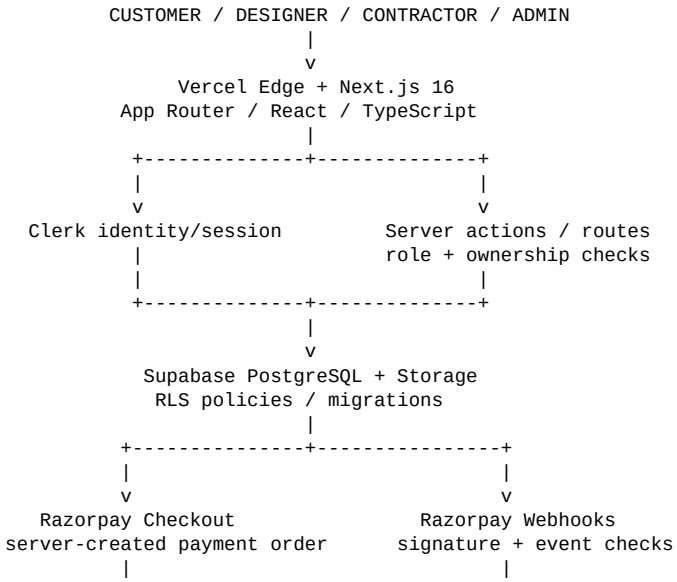
• No multi-vendor marketplace or commission workflow.

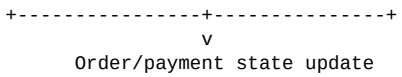
• No AI room rendering or native mobile app.

• Product checkout is in scope; project-service payments are deferred until separately approved.

3. High-level architecture

The application is a Next.js modular monolith. The Next.js app handles public pages, dashboards, server actions/route handlers, authorization checks, and payment endpoints. Clerk provides identity and sessions. Supabase provides PostgreSQL, Storage, and row-level data controls. Razorpay processes the checkout payment; verified webhooks update the order state.





Architecture decisions

• Use a modular monolith first: easier to build, test, and deploy for a student project than microservices.

• Use server components for data-heavy pages where suitable; client components only where interaction is needed.

• Put business rules in server-side service functions and validate inputs at boundaries.

• Use Supabase migrations as the source of truth for schema and security policies.

• Use the payment provider as payment processor; never store card numbers or CVV.

4. Authentication and role authorization

Clerk + Supabase is a supported pairing. Use Clerk's native Supabase integration / Supabase third-party auth provider flow. The legacy Clerk Supabase JWT-template approach is deprecated. Clerk authenticates the user; the application still has to authorize each action.

Recommended request flow

• User signs in through Clerk; the server obtains the verified Clerk identity.

• Application profile row maps the Clerk user ID to one of the four roles.

• Protected server action checks identity, role, and record ownership/assignment before mutation.

• When querying Supabase through user-scoped access, pass the Clerk session token according to current official integration docs.

• Enable RLS on exposed tables and write policies using verified JWT claims / Clerk user ID.

• Keep privileged server credentials server-only; never expose service-role/secret keys to the browser.

Role assignment

Public signups default to customer. Users must not be able to promote themselves to admin, designer, or contractor through editable form fields or untrusted metadata. Role changes are admin-only, server-validated, and ideally audited. Do not rely on navigation hiding as a security control.

5. Logical data model

Table	Purpose / important fields
profiles	clerk_user_id (unique text), display_name, email, role constrained to customer/designer/contractor/admin.
design_requests	customer_id, assigned_designer_id, title, project_type, area, budget range, requirements, status.
proposals	design_request_id, designer_id, scope, price_paise, valid_until, status, revision.
projects	design_request_id, proposal_id, customer_id, designer_id, status, dates.
tasks	project_id, assigned_contractor_id, title, status, due_date, progress_percent.
project_updates	project_id, author_id, message, attachment_path, created_at.
products	sku, slug, name, category, price_paise, stock_quantity, is_active.
product_images	product_id, storage_path, alt_text, sort_order.

Table	Purpose / important fields
orders	customer_id, order_number, totals in paise, status, Razorpay order ID.
order_items	order_id, product_id, product_name_snapshot, unit_price_paise, quantity, line_total_paise.
payments	order_id, provider IDs, amount_paise, currency, status, signature_verified.
webhook_events	unique provider_event_id, event type, processing status, processed timestamp.

Data rules

• Use UUIDs for application records; store Clerk user IDs as text.

• Store money as integer paise (e.g., ■499.00 = 49900), never floating-point values.

• Use foreign keys, uniqueness constraints, indexes, non-negative checks, and version-controlled migrations.

• Snapshot product name and unit price in order_items so historic orders remain accurate after catalogue edits.

• Use private storage for project documents; generate short-lived signed URLs for authorized access.

6. Key workflows

A. Consultation → proposal → project

```
Customer submits design request
   -> request status = submitted / under_review
Designer reviews and prepares proposal
   -> proposal linked to request; status = sent
Customer accepts or rejects
   -> reject: record decision
   -> accept: transactionally/idempotently create project
Designer creates milestones/tasks
   -> contractor assigned to specific tasks
Contractor updates progress
   -> customer/designer sees authorized project timeline
Project reaches completion
   -> designer/admin marks project completed
```

B. Product purchase

```
Cart submitted by customer
   -> server re-reads product prices and stock
   -> validates quantities and computes total in paise
   -> creates pending local order + Razorpay order
Browser opens Razorpay Checkout
   -> callback sends payment IDs/signature to server
Server verifies signature and payment/order status
Webhook also verifies provider event and is idempotent
   -> payment/order state changes to paid only when verified
   -> stock adjustment happens safely once
Customer order page reads server-side order state
```

Payment requirements

• Create every payment order on the server; never trust browser-supplied total.

• Verify Razorpay signature server-side using the secret key.

• Use webhook event IDs / idempotency controls to prevent duplicate processing.

• Do not fulfil an order until payment status is verified/captured.

• Keep secret keys out of client bundles and never store card/CVV data.

7. Route and module map

```
Public:
```

```
  /                 /services             /consultation
  /shop             /shop/[slug]          /about

Shopping:
  /cart             /checkout             /orders
  /orders/[orderId]

Authentication:
  /sign-in          /sign-up

Dashboards:
  /dashboard/customer
  /dashboard/designer
  /dashboard/contractor
  /dashboard/admin
  /dashboard/requests
  /dashboard/proposals
  /dashboard/projects/[projectId]
  /dashboard/tasks
  /dashboard/products
  /dashboard/orders

Server endpoints (illustrative):
  POST /api/checkout/create-order
  POST /api/checkout/verify
  POST /api/webhooks/razorpay
```

8. Suggested project structure

```
src/
  app/                  # App Router pages/layouts/route handlers
  components/
    ui/                 # shared UI primitives
    layout/             # header, footer, dashboard shell
    forms/              # validated forms
    products/            # product cards/detail/cart UI
    projects/            # project/task/progress UI
  lib/
    auth/                # Clerk identity helpers
    supabase/            # user-scoped/server clients
    permissions/         # role + ownership checks
    validation/          # Zod schemas
    payments/            # Razorpay integration
  server/
    actions/             # validated server actions
    services/            # business workflows
  types/
supabase/
  migrations/
  seed.sql
tests/
  unit/
  e2e/
```

For Next.js 16, follow current Clerk guidance using proxy.ts (not older middleware.ts examples). Confirm current SDK conventions before coding.

9. Security model

Role	Allowed scope (summary)	Must not do
Customer	Own requests, proposals, projects, orders; public active products.	Read/change another customer's records or admin data.
Designer	Assigned requests/projects, related proposals/tasks and updates.	Access unrelated customers/projects or change store payments.
Contractor	Assigned tasks and minimum necessary project context.	Change role, pricing, orders, or unassigned tasks.
Admin	Platform operations after server-side admin check.	Expose credentials or bypass audit/security controls.

• Server-side auth + role + ownership check for every protected mutation.

• RLS enabled for every table exposed via Supabase APIs; test policies with all roles.

• Validate inputs, upload size/type, and IDs. Use rate limits for public/payment endpoints where feasible.

• Private storage buckets and short-lived signed URLs for project files.

• No secrets in NEXT_PUBLIC_ variables except intended publishable/public keys.

• Avoid caching user-specific responses or pages in a way that leaks data.

• Audit admin role changes, catalogue changes, and order/payment interventions.

10. Deployment and operations

Target Vercel with a GitHub-connected project. Use Vercel Preview deployments for each branch/PR and Production for the main branch. Next.js is a first-class Vercel framework; use the normal Next.js build rather than static export because the project requires server-side auth, database access, checkout endpoints, and webhooks.

Build and deploy checklist

• Use Node.js 24.x if available in the Vercel project settings and compatible with selected packages.

• Commit package-lock.json / pnpm-lock.yaml and use the same package manager locally and in CI.

• Run npm run build before deployment; fix TypeScript and build errors instead of suppressing them.

• Configure Clerk, Supabase, and Razorpay variables separately for Preview and Production.

• Set Razorpay webhook URL to the deployed endpoint and configure the webhook secret.

• Use test-mode payment credentials in Preview until end-to-end tests pass.

• Run database migrations deliberately; keep schema changes version-controlled.

• Test sign-in, role boundaries, request/proposal workflow, checkout success/failure, webhook retries, and order history on the Preview deployment.

Environment variable names (illustrative; verify SDK docs)

```
NEXT_PUBLIC_CLERK_PUBLISHABLE_KEY=
CLERK_SECRET_KEY=
NEXT_PUBLIC_SUPABASE_URL=
NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY=
RAZORPAY_KEY_ID=
RAZORPAY_KEY_SECRET=
RAZORPAY_WEBHOOK_SECRET=
```

Never commit real values. Keep Clerk secret, Razorpay secret, and webhook secret server-only. Do not put secrets in screenshots, source code, issue reports, or AI prompts.

11. Implementation sequence

Phase 0 — Inspect

Inspect repository, detect existing framework, verify stable versions and agree on architecture. Do not rewrite existing work without approval.

Phase 1 — Foundation

Next.js/TypeScript/Tailwind, Clerk, native Supabase integration, profiles/roles, migrations, RLS, protected dashboards.

Phase 2 — Design workflow

Consultation requests, designer assignment, proposals, acceptance/rejection, projects, tasks, progress updates.

Phase 3 — Store

Admin catalogue, image storage, public listing/detail pages, cart, server-side price/stock validation.

Phase 4 — Payments

Razorpay server-created order, signature verification, webhook idempotency, order history/admin order management.

Phase 5 — Quality/deploy

Tests, accessibility, responsive polish, security review, Vercel Preview test, Production deployment.

12. Definition of done

- Customer cannot access another customer's data by guessing/changing record IDs.
- Designer and contractor access is restricted to assigned work.
- Public user cannot self-promote to a privileged role.
- Proposal acceptance cannot create duplicate projects on retry.
- Product price and stock are revalidated on the server at checkout.
- Invalid signatures and unverified payments cannot mark orders paid.
- Repeated webhooks cannot duplicate payment/order/stock operations.
- The build passes with the committed lockfile and the app works on Vercel Preview.
- Core design workflow is the main navigation priority; shopping remains complementary.

13. Official references

Check these sources again when implementation starts; framework and SDK releases can change.

- Next.js support policy — https://nextjs.org/support-policy
- Next.js deployment documentation — https://nextjs.org/docs/app/getting-started/deploying
- React versions — https://react.dev/versions
- Clerk Next.js quickstart — https://clerk.com/docs/nextjs/getting-started/quickstart
- Supabase — Clerk third-party auth — https://supabase.com/docs/guides/auth/third-party/clerk
- Clerk — Supabase integration — https://clerk.com/docs/guides/development/integrations/databases/supabase
- Tailwind CSS with Next.js — https://tailwindcss.com/docs/installation/framework-guides/nextjs
- Vercel Node.js versions — https://vercel.com/docs/functions/runtimes/node-js/node-js-versions
- Razorpay Standard Checkout integration — https://d6xcmfyh68wv8.cloudfront.net/docs/payments/payment-gateway/web-integration/standard/build-integration/